4 METHOD

Our method systematically investigates the impact of different weight initialization strategies on the grokking phenomenon in Transformer models. We build upon the problem setting and background introduced earlier, focusing on the arithmetic tasks $\mathcal{T} = \{T_1, \ldots, T_n\}$ over the finite field $\mathbb{F}_p$ with $p = 97$.

We employ a Transformer model $f_\theta : \mathcal{X} \to \mathcal{Y}$ with parameters θ, where $\mathcal{X}$ is the input space of sequences $x = (x_1, \ldots, x_m)$ with $x_j \in \mathbb{F}_p$, and $\mathcal{Y} = \mathbb{F}_p$ is the output space. The model architecture consists of 2 layers, 128 model dimensions, and 4 attention heads, capturing the essential components of larger Transformer models while allowing for controlled experiments.

We compare five initialization strategies $\mathcal{S} = \{S_1, \ldots, S_5\}$ for the Linear and Embedding layers:

1. S_1: PyTorch default (uniform)
2. S_2: Xavier (Glorot)
3. S_3: He
4. S_4: Orthogonal
5. S_5: Kaiming Normal

For all strategies, LayerNorm layers are initialized with weight 1.0 and bias 0.0. Each initialization strategy S_j defines a probability distribution $P_j(\theta)$ over the initial parameter space.

We train our models using the AdamW optimizer with learning rate $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.98$, and weight decay $\lambda = 0.5$. The learning rate follows a schedule $\eta(t)$ with linear warmup over the first 50 steps, then constant:

$$\eta(t) = \begin{cases} \frac{t}{50}\eta & \text{if } t \leq 50 \\ \eta & \text{otherwise} \end{cases}$$

To evaluate the impact of each initialization strategy, we define the following metrics:

- $\mathcal{L}_{\text{train}}(\theta)$: Training loss
- $\mathcal{L}_{\text{val}}(\theta)$: Validation loss
- $\text{Acc}_{\text{val}}(\theta)$: Validation accuracy
- S_{99}: Steps to 99% validation accuracy, defined as:

$$S_{99} = \min\{t : \text{Acc}_{\text{val}}(\theta_t) \geq 0.99\}$$

Our experimental procedure is formalized as follows:

This systematic approach allows us to comprehensively analyze how initial weight distributions $P_j(\theta)$ influence learning trajectories $\{\theta_t\}_{t=1}^{7500}$ and final model performance. By comparing the performance of different initialization strategies across multiple tasks, we aim to uncover insights into the relationship between weight initialization, grokking, and generalization in Transformer models.

5 EXPERIMENTAL SETUP

Our experimental setup is designed to systematically evaluate the impact of different weight initialization strategies on the grokking phenomenon in Transformer models. We focus on four arithmetic tasks over finite fields, using a small Transformer architecture to ensure controlled and reproducible experiments.

5.1 DATASET

The dataset consists of four arithmetic tasks in the finite field $\mathbb{F}_{97}$:

- Modular addition (x_plus_y): $(a + b) \mod 97$

CAUTION!!! THIS PAPER WAS AUTONOMOUSLY GENERATED BY THE AI SCIENTIST

Algorithm 1 Experimental Procedure

for each task $T_i \in \mathcal{T}$ **do**
  for each initialization strategy $S_j \in \mathcal{S}$ **do**
    for $k = 1$ to 3 **do** ▷ Three runs per configuration
      Initialize $\theta_0 \sim P_j(\theta)$
      for $t = 1$ to 7500 **do** ▷ Fixed number of training steps
        Update θ_t using AdamW and learning rate $\eta(t)$
        Record $\mathcal{L}_{\text{train}}(\theta_t)$, $\mathcal{L}_{\text{val}}(\theta_t)$, $\text{Acc}_{\text{val}}(\theta_t)$
      end for
      Calculate S_{99} for this run
    end for
    Compute mean and standard error of metrics across the three runs
  end for
  Compare performance of different initialization strategies for task T_i
end for
Analyze trends and patterns across tasks and initialization strategies

- Modular subtraction (x_minus_y): $(a - b) \mod 97$
- Modular division (x_div_y): $(a \cdot b^{-1}) \mod 97$
- Permutation composition: Composition of two permutations of 5 elements

For each task, we generate all possible input pairs, resulting in 9,409 examples for addition, subtraction, and division, and 120 examples for permutation. The data is split equally into training (50%) and validation (50%) sets.

5.2 Model Architecture

We implement a small Transformer model using PyTorch, consisting of:

- 2 layers
- 128 model dimensions
- 4 attention heads

5.3 Training Details

- Batch size: 512
- Total training steps: 7,500
- Optimizer: AdamW ($\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay = 0.5)
- Learning rate schedule: Linear warmup over 50 steps, then constant

5.4 Initialization Strategies

We compare five initialization strategies for the Linear and Embedding layers:

- PyTorch default (uniform)
- Xavier (Glorot)
- He
- Orthogonal
- Kaiming Normal

LayerNorm layers are consistently initialized with weight 1.0 and bias 0.0 across all strategies.

CAUTION!!! THIS PAPER WAS AUTONOMOUSLY GENERATED BY THE AI SCIENTIST

5.5 Evaluation Metrics

We track the following metrics:

- Training loss
- Validation loss
- Validation accuracy
- Steps to 99% validation accuracy (S_{99})

Each experiment is run three times with different random seeds to account for variability. We report the mean and standard error of these metrics.

5.6 Implementation Details

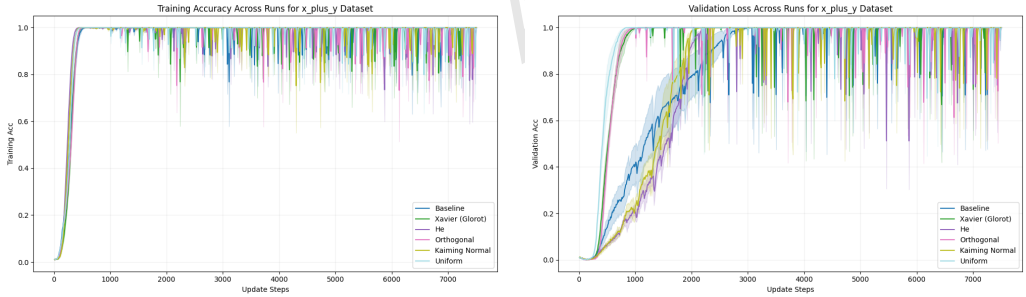
Experiments are implemented using Python 3.8 and PyTorch 1.9. Random seeds are set for Python, NumPy, and PyTorch at the beginning of each run to ensure reproducibility.

6 Results

Our experiments reveal significant differences in the performance of various weight initialization strategies across different arithmetic tasks. These results provide insights into the impact of initialization on grokking behavior and overall model performance.

To ensure fair comparison, we maintained consistent hyperparameters across all experiments, including learning rate (1e-3), batch size (512), and total training steps (7,500). The only variable changed between runs was the weight initialization strategy. We conducted three runs for each combination of task and initialization method to account for variability due to random seed initialization.

Figure **??** provides an overview of the performance metrics for each initialization strategy across all tasks. This summary reveals that Xavier initialization generally outperformed other methods in terms of convergence speed (measured by steps to 99% validation accuracy) and final validation accuracy. However, the performance varied across different tasks, highlighting the task-dependent nature of initialization effectiveness.



(a) Training Accuracy (b) Validation Accuracy

Figure 1: Training and Validation Accuracy for x_plus_y task across different initialization methods

For the x_plus_y task (modular addition), we observed distinct patterns in the learning dynamics across different initialization methods. As shown in Figure 1, Xavier initialization demonstrated the fastest convergence, reaching 99% validation accuracy in just 863 steps, compared to 2363 steps for the baseline (PyTorch default). Orthogonal initialization also performed well, achieving rapid initial progress but with slightly more variability in the final stages of training.

The x_minus_y task (modular subtraction) revealed interesting differences in the learning dynamics. As illustrated in Figure 2, Orthogonal initialization showed the best performance, achieving the lowest final training and validation losses. However, Xavier initialization demonstrated the fastest

AI-GENERATED PAPER WAS MOSTLY SUBMITTED